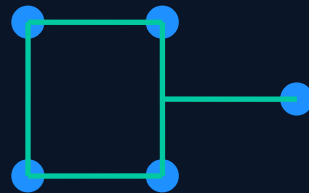


Linguagens de Programação

Noções Preliminares — Unidade I

Prof. Anderson Resende Lamas

andersonlamas@univicsa.com.br



"Uma linguagem que não afeta a maneira como você pensa sobre programação, não vale a pena aprender."

— Alan Perlis, 1º laureado com o Turing Award

Agenda da Unidade I

01 Por que estudar conceitos de LP?

02 Domínios de Programação

03 Critérios de Avaliação de Linguagens

04 Legibilidade — Fatores Detalhados

05 Facilidade de Escrita e Confiabilidade

06 Custo e Influências no Design de LP

07 Categorias de Linguagens e Trade-offs

08 Métodos de Implementação

09 Ambientes de Programação

S E Ç Ã O 0 1

Por que estudar LP?

Motivação, benefícios práticos e impacto no avanço da computação

Razões para estudar conceitos de Linguagens de Programação



Expressar Melhor Ideias

Assim como o vocabulário rico melhora a comunicação humana, conhecer paradigmas e construções permite formular soluções de forma mais precisa, elegante e eficiente.



Escolher a LP Adequada

Programadores tendem a usar sempre as mesmas ferramentas, mesmo quando inadequadas. Conhecer os fundamentos permite selecionar a linguagem mais adequada para cada contexto e projeto.



Aprender Novas Linguagens

Compreender conceitos fundamentais acelera o aprendizado de qualquer nova linguagem. Muitos princípios são compartilhados entre linguagens diferentes.



Entender a Implementação

Saber por que uma linguagem foi projetada de determinada forma ajuda a evitar armadilhas, a entender erros e a usar os recursos corretamente.



Melhor Uso do que Conhece

Estudar conceitos revela recursos desconhecidos nas linguagens que você já usa, aumentando produtividade e qualidade do código produzido.



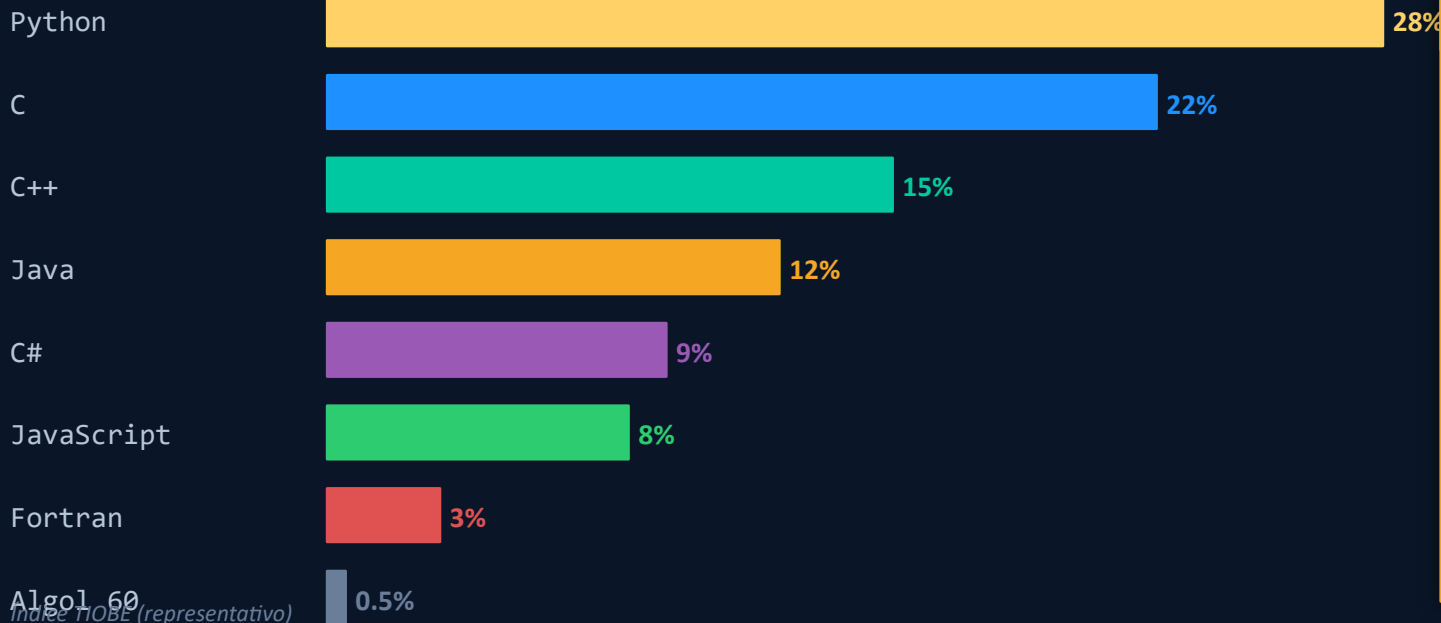
Avançar a Computação

Linguagens populares nem sempre são as tecnicamente superiores. Conhecimento crítico permite influenciar melhores escolhas no ecossistema tecnológico.

Popularidade ≠ Superioridade Técnica

Caso histórico — Algol 60 vs Fortran

Algol 60 tinha design conceitual superior (blocos estruturados, recursão), mas foi preterido porque poucos entendiam seus benefícios.



Conclusão

Linguagens dominam por fatores históricos, políticos e econômicos — nem sempre pelo mérito técnico.

Entender LP evita que más escolhas se perpetuem por inércia.

SEÇÃO 02

Domínios de Programação

Aplicações Científicas · Empresariais · IA · Web

Aplicações Científicas e Empresariais



Aplicações Científicas

Contexto histórico

Primeiros computadores (anos 1940–50) foram desenvolvidos exclusivamente para aplicações científicas e militares — cálculo balístico, projeto de bombas, tabelas aerodinâmicas.

Estruturas necessárias



Vetores e matrizes (estruturas de dados)



Laços de contagem (estruturas de controle)



Aritmética de ponto flutuante de alta precisão

FORTRAN (1957) — John Backus / IBM

Ainda em uso hoje; nenhuma LP após 1960 é significativamente mais eficiente em computação numérica.



Aplicações Empresariais

Requisitos específicos

Sistemas empresariais exigem processamento em lote, geração de relatórios complexos, manipulação precisa de decimais e formatação de saída orientada a negócios.

Recursos necessários



Processamento de registros e arquivos



Geração de relatórios formatados



Aritmética decimal exata (sem erro de ponto flutuante)

COBOL (1960) — Grace Hopper / Dept. of Defense

Provavelmente ainda é a LP mais usada para sistemas bancários. Movimenta +\$3 trilhões/dia.

Inteligência Artificial — Computação Simbólica

IA é caracterizada pelo uso de computações simbólicas em vez de numéricas — manipulação de símbolos, inferência lógica, aprendizado por regras e representação de conhecimento.

LISP (1959)

John McCarthy / MIT

Primeiro LP para IA. Introduziu listas encadeadas, recursão automática, coleta de lixo (GC) e funções como cidadãos de primeira classe. Base conceitual para linguagens funcionais modernas.

LISP

```
(defun fatorial (n)
  (if (<= n 1) 1
      (* n (fatorial (- n 1)))))

(fatorial 5) ; → 120
```

Prolog (1972)

Alain Colmerauer / Univ. Aix-Marseille

Paradigma lógico-declarativo. O programador define fatos e regras; o motor de inferência decide como resolvê-los. Muito usado em sistemas especialistas e processamento de linguagem natural.

Prolog

```
pai(tom, bob).
pai(bob, ann).

avô(X, Z) :- pai(X, Y), pai(Y, Z).

?- avô(tom, ann). % → true
```


Software para Web — Da Marcação ao Backend



Apresentação

HTML5 + CSS3

Linguagem de marcação (não é LP). Define estrutura semântica e estilo visual das páginas. Não possui lógica computacional.



Interatividade (Front-end)

JavaScript / TypeScript

LP de scripting interpretada pelo browser. Permite conteúdo dinâmico, manipulação do DOM e comunicação assíncrona (AJAX/Fetch). TypeScript adiciona tipagem estática.



Lógica de Negócio (Back-end)

PHP / Python / Node.js / Java

Processamento no servidor, acesso a banco de dados, autenticação, APIs REST. Cada tecnologia tem trade-offs de desempenho, escalabilidade e produtividade.



Banco de Dados

SQL / NoSQL

SQL (MySQL, PostgreSQL) para dados relacionais; NoSQL (MongoDB, Redis) para dados não estruturados ou de alta performance. SQL é uma LP declarativa própria.

SEÇÃO 03

Critérios de Avaliação

Legibilidade · Facilidade de Escrita · Confiabilidade · Custo

VISÃO GERAL DOS CRITÉRIOS

CARACTERÍSTICA	LEGIBILIDADE	FACILIDADE DE ESCRITA	CONFIABILIDADE
Simplicidade	✓	✓	✓
Ortogonalidade	✓		✓
Tipos de dados	✓	✓	✓
Projeto de sintaxe	✓	✓	✓
Suporte p/ abstração		✓	✓
Expressividade		✓	✓
Verificação de tipos			✓
Tratamento de exceções			✓
Apelidos restritos			✓

Como avaliar uma Linguagem de Programação?

Legibilidade

Facilidade com que os programas podem ser lidos e entendidos.

Características:

- Simplicidade
- Ortogonalidade
- Tipos de dados
- Projeto de sintaxe

Facilidade de Escrita

Quão facilmente uma linguagem permite criar programas para um domínio específico.

Características:

- Simplicidade
- Ortogonalidade
- Expressividade
- Suporte à abstração

Confiabilidade

O programa se comporta conforme especificado em todas as condições.

Características:

- Verificação de tipos
- Tratamento de exceções
- Apelidos restritos

S E Ç Ã O 0 4

Legibilidade

Simplicidade · Ortogonalidade · Tipos · Sintaxe

Legibilidade — Definição e Contexto Histórico

Legibilidade: facilidade com que os programas podem ser lidos e entendidos por humanos.

Anos 1950–60

Foco na Máquina

Eficiência era a virtude principal. O código era escrito para o processador, não para o programador. Legibilidade era secundária — o que importava era velocidade de execução.

Anos 1970

Virada: Manutenção

O conceito de ciclo de vida de software revelou que o custo de manutenção supera o de desenvolvimento. Legibilidade tornou-se o critério mais importante. Orientação à máquina → orientação às pessoas.

Anos 1980–90

Orientação a Objetos

Encapsulamento, herança e polimorfismo introduziram novas dimensões de legibilidade. Java e C++ trouxeram código mais longo, porém mais organizado e reutilizável.

Hoje

Clean Code e Legibilidade

Robert C. Martin popularizou 'Clean Code'. Python foi projetado explicitamente para máxima legibilidade. TypeScript corrige a falta de tipos do JavaScript, melhorando legibilidade e manutenção.

Simplicidade Geral — Três Problemas

Uma linguagem com muitas construções básicas é mais difícil de aprender. Programadores tendem a aprender apenas um subconjunto e ignorar o restante — criando inconsistências no código de equipes.

① Multiplicidade de Recursos

Múltiplas formas de realizar a mesma operação geram confusão e inconsistência no código de equipes.

C — 4 formas equivalentes

```
count = count + 1;   count += 1;
count++;             ++count;
```

② Sobrecarga de Operadores

Um operador com múltiplos significados pode ser útil (+ para int e float) ou perigoso (+ para vetores).

C

```
int a = 3+5;    // 8   ✅ previsível
Vetor C = A+B;  // 😬 ambíguo
```

③ Simplicidade Extrema — Assembly

Instruções muito simples exigem muitas linhas de código — dificultando o entendimento do fluxo geral do programa.

Assembly: 4 linhas para 1 soma

```
MVI B, 05h ; B = 5
MVI C, 01h ; C = 1
MOV A, B   ; A = B
ADD C      ; A = A + C → int x = 5+1;
```

Equivalente em C:

C

```
int x = 5 + 1;
// Uma linha clara vs 4 instruções assembly
```

Ortogonalidade — Combinações Consistentes e Previsíveis

Ortogonalidade:

pequeno número de primitivas combinadas de forma consistente, previsível, sem exceções arbitrárias.

Origem matemática: vetores ortogonais são independentes entre si — mudando um, o outro não é afetado.

4 tipos × 2 operadores = 8 combinações todas válidas:

	int	float	double	char
Vetor	✓ Vetor[int]	✓ Vetor[float]	✓ Vetor[double]	✓ Vetor[char]
Ponteiro	✓ Ponteiro[int]	✓ Ponteiro[flloat]	✓ Ponteiro[doubl le]	✓ Ponteiro[char]

⚠ Falta de ortogonalidade em C

```
C // ✗ Vetores: NÃO podem ser retornados
int[] f() { ... } // INVÁLIDO em C

// ✓ Structs: podem ser retornados
struct Pt f() { ... } // VÁLIDO

// Mesma posição sintática → comportamentos
// completamente diferentes = não ortogonal
```

Assembly IBM × VAX — diferença de ortogonalidade

ASM

```
; IBM — instruções separadas por tipo
A   Reg1, mem ; reg + memória
AR  Reg1, Reg2 ; reg + reg

; VAX — instrução única, mais ortogonal
ADDL op1, op2 ; qualquer combinação
```


Tipos de Dados e Projeto de Sintaxe



Tipos de Dados Adequados

A ausência de tipos expressivos força o programador a usar 'números mágicos' que não comunicam intenção.

```
// Sem booleano – significado obscuro
timeout = 1; // Significa 'true'? Erro?
```

```
// Com booleano – intenção clara
timeout = true; // ✅ Autoexplicativo
status = ATIVO; // ✅ Enum melhora ainda mais
```



Projeto de Sintaxe

Escolhas de sintaxe impactam diretamente na legibilidade. C usa chaves (menos legível), Pascal usa palavras (mais legível).

```
// Pascal – mais legível
if x > 10 then
  writeln('Maior');
end if;
```

```
// C – chaves dificultam rastrear escopo
```

⚠️ Forma ≠ Significado — Palavra reservada 'static' em C

C – static local

```
// Dentro de uma função:
void f() {
  static int contador = 0; // variável
  contador++;              // persiste entre
}                          // chamadas!
```

C – static global

```
// Fora de funções (global):
static int variavel = 10;
// Visibilidade restrita ao arquivo
// MESMO nome, significado OPOSTO!
// → viola forma e significado
```

SEÇÃO 05

Facilidade de Escrita e Confiabilidade

Expressividade · Verificação de Tipos · Exceções · Apelidos

Facilidade de Escrita — Escrever para o Domínio Certo

Facilidade de escrita: medida de quão facilmente uma linguagem pode ser usada para criar programas para um domínio específico. Não é justo comparar linguagens projetadas para domínios diferentes.

Simplicidade e Ortogonalidade

Se a linguagem tem um grande número de construções, alguns programadores não as conhecerão todas, podendo usar recursos incorretamente. Um conjunto menor e mais consistente de regras — ortogonalidade — produz código melhor.

```
# Python: ortogonalidade alta
lista = [1,2,3,4]
resultado = [x**2 for x in lista if x>1]
# List comprehensions funcionam com qualquer
# tipo iterável, de forma consistente
```

Suporte à Abstração

Capacidade de definir e usar estruturas complexas ignorando os detalhes de implementação. Abstração de processos (sub-programas) e de dados (tipos abstratos) são fundamentais para escrever programas maiores.

```
# Abstraíndo o conceito de Fila
from collections import deque
fila = deque()
fila.append('cliente1') # enfileira
fila.popleft()          # desenfileira
# Não preciso saber como é implementada
```

Expressividade

Recursos que tornam a escrita mais conveniente e concisa. Exemplo: `count++` é mais conciso que `count = count + 1`. O uso de `for` ao invés de `while` em laços de iteração sobre coleções é mais expressivo.

```
// Java – for-each expressivo
for (String nome : lista) {
    System.out.println(nome);
}
// C com while – menos expressivo
int i=0;
while(i<n) { printf("%s\n", lista[i++]); }
```

Visual Basic × C para GUIs: VB foi projetado para isso, C não. O domínio da aplicação deve guiar a escolha da linguagem.

Confiabilidade — Verificação de Tipos

Verificação de tipos: execução de testes para detectar erros de tipo — por parte do compilador (preferível) ou durante a execução. A verificação em tempo de compilação é mais barata e eficiente.

✓ Tempo de Compilação (Estática)

Erros detectados antes de executar. Mais barato corrigir. Exemplos: Java, C, C++, Haskell.

Vantagem: programas mais rápidos (sem verificações em runtime) e mais confiáveis.

Java

```
// Java – verificação estática
String s = 42; // ERRO de compilação

void f(float x) { ... }
f(23); // 23(int) → problema detectado
```

⚠ Tempo de Execução (Dinâmica)

Erros só aparecem quando o código problemático é executado. Pode passar despercebido por anos. Exemplos: Python (dinâmico), JavaScript, PHP.

Desvantagem: erro em produção pode causar falha grave.

Python

```
# Python – verificação dinâmica
x = 'texto'
resultado = x + 5 # TypeError:
# só aparece em runtime!
```

IEEE 754: `int 23 = 0x00000017` ≠ `float 23.0 = 0x41B80000` — representações binárias completamente diferentes; misturá-las sem conversão gera resultados sem sentido.

Tratamento de Exceções e Apelidos (Aliases)

Tratamento de Exceções: habilidade de interceptar erros em tempo de execução, tomar medidas corretivas e continuar.

Python – try/except

```
try:
    num = int(input('Digite um número: '))
    resultado = 10 / num # possível div/0
    print(f'Resultado: {resultado}')
except ZeroDivisionError:
    print('Erro: divisão por zero!')
except ValueError:
    print('Entrada inválida. Digite inteiro.')
finally:
    print('Fim do programa.')
```

Sem tratamento de exceção:

O programa falha completamente ao encontrar um erro não previsto, possivelmente corrompendo dados e deixando recursos em estado inconsistente.

Com tratamento de exceção:

O programa pode recuperar-se de erros previsíveis, registrar logs, liberar recursos e continuar operando — aumentando robustez e confiabilidade.

Apelidos (Aliases): dois ou mais nomes para a mesma célula de memória — recurso perigoso, pois alterar por um afeta o outro.

C – Definição de aliases

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p1 = &x; // p1 → x
    int *p2 = &x; // p2 → x (mesmo endereço!)
    *p1 = 20; printf("x=%d\n",x); // x=20
```

⚠ Por que é perigoso?

Modificar x via p2 afeta silenciosamente tudo que usa p1. Em sistemas complexos, isso causa bugs extremamente difíceis de rastrear.

Java evita ponteiros diretos precisamente para

Ada foi pioneira em tratamento de exceções estruturado. C++ e Java popularizaram try/catch/finally. Rust elimina toda uma classe de bugs de memória em tempo de compilação.